



Practica 8

Practica8_2026.pdf

Ejercicio 1: Una sentencia puede ser simple o compuesta, ¿Cuál es la diferencia?

- Sentencia simple → Instrucción básica que realiza una sola acción y no contiene más sentencias anidadas. Se realiza de manera secuencial y corresponde a una línea de código que hace algo específico
- Sentencia compuesta → Conjunto de sentencias agrupadas, se agrupan varias simples o varias compuestas en una con el uso de delimitadores (Begin/End en Ada y Pascal, { } en C/C++/Java)

Ejercicio 2: Analice como C implementa la asignación.

- La asignación en C se hace mediante '=' y un ejemplo sería "a = 3 + 8;"
 - Siendo **a** una variable que representa a un identificador en memoria donde se almacena el valor
 - 3 + 8** es una expresión que puede ser constante, una variable, una operación, una llamada a una función y el resultado se evalúa a un valor compatible con el tipo de variable
- Las sentencias de asignación devuelven valores y se define la sentencia de asignación como una expresión con efectos colaterales
- C evalúa de derecha a izquierda → a=b=c=0, se asigna 0 a "c", "c" a "b", "b" a "a"
- Si se usa asignaciones en condiciones primero asigna y luego evalúa, tomando como verdadero todo resultado distinto de 0

Ejercicio 3: ¿Una expresión de asignación puede producir efectos laterales

que afecten al resultado final, dependiendo de cómo se evalúe? De ejemplos.

-Dependiendo el orden de evaluación de las subexpresiones esto puede ocurrir. Las asignaciones no solo almacenan valores sino que también pueden modificar variables que se usan en otras partes de la misma expresión

-Modificación de una variable usada en la misma expresión:

```
int x = 5;
int y = (x = 10) + x;
// x = 10 reasigna 10 a x,
// Se suma x de nuevo que ahora vale 10
// y = 10 + 10 = 20

// Efecto secundario -> La asignación x=10 termina cambiando el valor de X
// y afecta a la expresión
```

-Operadores de incremento con asignación:

```
int x = 5;
int y = x++ + (x = 10)

// x++ usa el valor actual, por lo tanto incrementa a 6
// x = 10 reasigna 10 por lo tanto toma un nuevo valor
// y = 5 + 10 + 15
// Si el orden de evaluación no está definido esto puede dar diferentes
// resultados ya que puede primero reasignarse x = 10 y cambiar todo
```

-Asignación encadenada:

```
int a = 1, b = 2;
a = b = a + 1

// Se evalúa a + 1 = 1 + 1 = 2
// Se asigna 2 a b
```

```
// Se vuelve a evaluar b que ahora es 2
// Se asigna "b" a "a"

// La asignacion a "b" depende del valor modificado de
a + 1
// Esto puede ser alterado en la misma expresion
```

Ejercicio 4: Qué significa que un lenguaje utilice circuito corto o circuito largo para la evaluación de una expresión. De un ejemplo en el cual por un circuito de error y por el otro no.

-La terminología circuito corto y circuito largo se refiere a como se evalúan las expresiones lógicas en ciertos contextos. Son técnicas utilizadas en la evaluación de expresiones booleanas (involucran AND y OR)

-Circuito corto → Los operandos de una expresión lógica se evalúan de izquierda a derecha hasta encontrar el primero que determina el resultado final. En ese punto la evaluación se detiene y no se evalúan los operandos restantes

-Circuito largo → Todos los operandos se evalúan sin importar si el primero ya determina el resultado final

-Ejemplo que da error y otro no

```
function buscarEnVector (v:Tvector; n: integer):boolean;
var
    i: integer;
begin
    i := 1;
    while (i <= dimF) and (v[i] <> n) do
        i := i + 1;
        buscarEnVector := (i <= dimF) // Retorna true a
        l encontrarlo y false si no
    end;
```

-Gracias al circuito corto esto no da error ya que se verifica que el índice *i* no supere el número de elementos que contiene el arreglo. En caso de que se supere la cantidad ni siquiera se evalúa la condición de que *v[i]*

<> n y se termina la ejecucion ahi. Si se tuviera un circuito largo se intentaria acceder a una posicion fuera de rango de arreglo al momento de evaluar la 2da condicion y daria error

Ejercicio 5: ¿Qué regla define Delphi, Ada y C para la asociación del else con el if correspondiente? ¿Cómo lo maneja Python?

-ADA y Delphi:

-Se utiliza la coincidencia de las palabras clave begin y end para determinar la asociacion del else con el if correspondiente mas cercano. El else se asocia con el if que tiene la misma palabra clave begin y end mas cercana y que no tiene un else asociado

-C:

-Cada rama else se empareja con la instruccion if solitaria mas proxima. Se empareja cada else para cerrar al ultimo if abierto

-Python:

-Usa indentacion para determinar el cuerpo de las estructuras, por lo que la asociacion del else con el if correspondiente se basa en esto mismo, buscando el if con la misma indentacion el else a anidar

Ejercicio 6: ¿Cuál es la construcción para expresar múltiples selección que implementa C? ¿Trabaja de la misma manera que la de Pascal, ADA o Python?

-C:

- Constructor Switch seguido de expresion
- Cada rama Case es etiquetada por uno o mas valores constantes (integer o char)
- Si coincide con una etiqueta del Switch se ejecutan las sentencias asociadas y se continua con las sentencias de las otras entradas (para eso hay que poner un break para que no revise todas)
- Existe la opcion default para los casos que el valor no coincida con ninguna de las opciones establecidas (opcional)
- Si un valor no cae dentro de alguno de los casos de la lista y no hay un default no provoca error pero provoca efectos colaterales dentro del codigo
- El orden de las ramas no importan

-Pascal:

- Se usa la palabra clave case seguida de una variable-expresion de tipo ordinal y la palabra of
- Variable-expresion a evaluar se llama selector
- Lista de sentencias para los diferentes valores que puede adoptar la variable que ademas llevan etiquetas
- El orden no importa
- Bloque else en caso de que la variable adopte un valor que no coincida con ninguna de las sentencias de la lista (opcional)
- Se coloca un end; (no corresponde con ningun begin que exista)
- Es inseguro ya que no determina que pasa si un valor no cae dentro de ninguna de las opciones de la lista

-ADA:

- Constructor: case expresion is con when y end case
- Las expresiones solo pueden ser de tipo entero o enumerativo
- El case debe contemplar todos los valores posibles que puede tomar la expresion, sino se genera un error en caso de que no coincida
- El when $\Rightarrow$ indica la accion/sentencia a ejecutar si se cumple la condicion
- end case; se usa siempre al final para el cierre
- When others (opcional) para representar aquellos valores que no caen en los explicitos previamente
- Una vez se ejecuta una rama del case, el case finaliza y no se ejecuta ninguna otra rama como en C
- No pasa la compilacion en caso de que no se coloque una rama para un posible valor y no aparece la opcion Others en esos casos

-Python:

- Constructor: match expresion: seguido de bloques case patron: identados
- Tipos de expresiones: pueden ser de cualquier tipo (enteros, cade,aslistas,objetos) y los patrones pueden ser literales, variables, estructuras de datos o patrones complejos

- Cobertura de casos: no es necesario contemplar todos los valores posibles de la expresion ya que is no hay conincidencia y no se usa "_" no hay error y no se ejecuta nada
- Indicador de accion: case patron: define el patron a comparar, seguido de un bloque indentado con las sentencias a ejecutar si el patron coincide
- Cierre, el fin de la estructura se indica por la desindentacion del codigo
- Caso por defecto: se usa case_ (opcional) y captura cualquier valor que no coincida con ningun patron de los anteriores. Este case debe ser el ultimo ya que cualquier case posterior es inalcanzable aunque no genera error si lo pones antes
- Ejecucion exclusiva: si coincide un case, se jecuta su bloque y el match termina, no se evaluan las otras ramas como en C

Ejercicio 7: Sea el siguiente código:

<pre> var i, z:integer; Procedure A; begin i:= i +1; end; begin z:=5; </pre>	<pre> for i:=1..5 do begin z:=z*5; A; z:=z + i; end; end; </pre>
--	--

a- Analice en las versiones estándar de ADA y Pascal, si este código puede llegar a traer problemas. Justifique la respuesta.

b- Comente qué sucedería con las versiones de Pascal y ADA, que Ud. utilizó.

- Este codigo podria traer problemas en Pascal ya que este mismo no permite que los valores del limite inferior y superior ni la variable de control se modifiquen. Dentro del bucle del procedure "A" se rompe con esta regla ya que estas modificando el indice utilizado en el For. Sin embargo, esta modificacion se realiza fuera de la misma unidad del For y no genera error
- En ADA el codigo no compila ya que se prohíbe modificar la variable de control "i" dentro del bucle for

Ejercicio 8: Sea el siguiente código en Pascal:

```
var puntos: integer;
begin
...
case puntos
1..5: write("No puede continuar");
10:write("Trabajo terminado")
end;
```

Analice, si esto mismo, con la sintaxis correspondiente, puede trasladarse así a los lenguajes ADA, C. ¿Provocarí error en algún caso? Diga cómo debería hacerse en cada lenguaje y explique el por qué. Codifíquelo.

-No cubre todos los casos ya que el valor entero puede tomar 0 o 6,7,8, etc caerian fuera del rango. Por ejemplo en ADA esto no compilaria ya que no hay una clausula when others para capturar valores que caen fuera del rango. En C tampoco se cubren los casos ya que no hay coincidencia y no hay sentencia default, ademas en C no podes usar el rango 1..5 porque generaria un error de sintaxis (cada case debe ser un valor individual)

-ADA:

```
with Ada.TEXT_IO; use Ada.TEXT_IO;
procedure Main is
  puntos: integer;
begin
  puntos := 10
  case puntos is
    when 1..5 =>
      Put_Line("No se puede continuar")
    when 10 =>
      Put_Line("Trabajo terminado")
    when others =>
      Put_Line("Valor no especificado")
```

```
        end case;  
    end Main;
```

-C:

```
#include <stdio.h>  
int main(){  
    int puntos = 10;  
    switch (puntos){  
        case 1:  
        case 2:  
        case 3:  
        ...  
        case 5:  
            printf("No se puede continuar")  
        case 10:  
            printf("Trabajo terminado")  
        default:  
            printf("Valor no especificado")  
        }  
    return 0;  
}
```

Ejercicio 9: Qué diferencia existe entre el generador YIELD de Python y el return de una función. De un ejemplo donde sería útil utilizarlo.

-Comportamiento:

- return → Termina la ejecución de la función inmediatamente y devuelve un único valor o nada si no se especifica. La función no se puede retomar después de un return
- yield → Convierte la función en un generador que produce una secuencia de valores uno a la vez. La ejecución se pausa en cada yield, permitiendo retomarla más tarde, ahorrando memoria al generar valores bajo demanda

-Estado:

- return → NO conserva el estado entre llamadas, cada invocación reinicia la función desde el principio

-yield → Mantiene el estado interno entre iteraciones, retomando desde donde se dejó

-Uso de memoria:

-return → Devuelve todos los datos de una vez, lo que puede consumir mucha memoria si el resultado es grande

-yield → Genera valores de forma iterativa

-Tipo de resultado:

-return → Devuelve un valor o estructura de datos

-yield → Devuelve un objeto generador que se puede iterar con un bucle o next

Ejercicio 10: Describa brevemente la instrucción map en javascript y sus alternativas.

-La instrucción map en javascript es una función de los arrays que crea un nuevo array aplicando una función callback a cada elemento del array original sin modificarlo (devuelve uno nuevo con los resultados)

```
// Ejemplo  
array.map(callback(elemento, indice, array) [, thisArg]).
```

-Alternativas:

-forEach:

-Itera sobre los elementos y ejecuta una función por cada uno pero no devuelve un nuevo array (solo hace los efectos secundarios)

-Ejemplo → numeros.forEach (num ⇒ resultado.push(num * 2))

-for:

-Permite modificar el array original o crear uno nuevo

-Ejemplo → for (let i = 0 ; i < num.length ; i++) { resultado.push (num[i] * 2); }

-for .. of:

-Itera sobre los elementos de un array de una forma más legible pero requiere crear manualmente un nuevo array

-Ejemplo → for (const num of numeros) { resultado.push(num * 2 ; }

Ejercicio 11: Determine si el lenguaje que utiliza frecuentemente implementa instrucciones para el manejo de espacio de nombres. Mencione brevemente qué significa este concepto y enuncie la forma en que su lenguaje lo implementa. Enuncie las características más importantes de este concepto en lenguajes como PHP o Python.

-Un espacio de nombres es un mecanismo que permite organizar código agrupando identificadores (nombres de clases, funciones, variables) bajo un nombre único, evitando conflictos entre nombres idénticos en diferentes partes de un programa

-Java → Implementa instrucciones para el manejo de espacio de nombres a través de paquetes. Los paquetes son el mecanismo principal para organizar y gestionar nombres en Java, evitando colisiones y estructurando el código

-PHP y Python:

-PHP: implementa espacios de nombres con la instrucción namespace

- **Declaración:**

```
php

namespace MiProyecto\SubEspacio;
class MiClase {}
```

- **Uso:**

- Importar con **use**:

```
php

use MiProyecto\SubEspacio\MiClase;
$obj = new MiClase();
```

Ámbitos globales y locales: \ al inicio indica el espacio global (e.g., strlen()).

Alias: use MiProyecto\SubEspacio\MiClase as Alias; permite renombrar para evitar conflictos.

Soporte dinámico: Los nombres pueden resolverse en tiempo de ejecución.

Evita colisiones: Útil en proyectos grandes o al usar bibliotecas de terceros (e.g., Composer).

-Python: no cuenta con una instrucción explícita para el espacio de nombres, pero los implementa con módulos y paquetes

- **Declaración:**
 - Un módulo es un archivo `.py` (e.g., `modulo.py`).
 - Un paquete es un directorio con un archivo `__init__.py`.
 - Ejemplo de estructura:

```

text

mi_paquete/
  __init__.py
  modulo.py

```
- **Uso:**

```

python

import mi_paquete.modulo
from mi_paquete.modulo import MiClase

```

Espacios implícitos: Cada módulo crea su propio espacio de nombres.

Importaciones relativas: `from .modulo import MiClase` dentro de un paquete.

Evita colisiones: `mi_paquete.modulo.MiClase` y `otro_paquete.modulo.MiClase` pueden coexistir.

Flexibilidad: Los módulos son objetos dinámicos, permitiendo inspección y manipulación en tiempo de ejecución.

fotos robadas de Guaymas pq alta paja buscarlo